PROJECT REPORT

SyncUp: Remote Team Daily Stand-up Tool

A Full-Stack MERN Application for Distributed Team Management

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology in Computer Science and Engineering

Presented by:

B. Supraja Chowdary – AP24110010750
V. Sarvagna – AP24110010788
G. Sai Pravachana – AP24110010795
Ch. Bhavya Sri – AP24110010825
D. Pragna – AP24110010836

Under the Guidance of:
Mr. Sidhartha Chennuri

Academic Year: 2025 – 2026

INTRODUCTION

SyncUp is a versatile and professional full-stack web application designed to streamline asynchronous communication and project tracking for distributed teams. Tailored specifically for the modern remote work environment, SyncUp offers a structured system for daily progress reporting — including detailed task logs, blocker identification, and multi-image attachments — that can be managed through a refined and intuitive interface.

Users can operate within specialized roles such as Admin, Team Lead, or Member, ensuring that project management is organized, role-appropriate, and secure. SyncUp features a sophisticated "Executive Linen" nude palette that prioritizes a clean and professional aesthetic, significantly reducing cognitive fatigue during long work sessions.

Robust security features — including JWT (JSON Web Token) authentication and Bcrypt password hashing — ensure data integrity and private workspace access at all times. SyncUp also integrates Real-Time WebSockets through Socket.io for instant workspace coordination and maintains a persistent Team Feed for historical data analysis and stakeholder reporting.

Its focus on clarity, real-time updates, and visual proof of work makes SyncUp an ideal choice for software firms, creative agencies, and global enterprises seeking an efficient and professional alternative to time-consuming synchronous meetings.

SCENARIO BASED CASE STUDY

Meet Maya, a Senior Software Project Coordinator with a demanding global schedule. Maya is responsible for managing a cross-functional team of developers, designers, and quality assurance testers located across four different time zones. Her primary challenge is keeping track of daily progress and identifying project bottlenecks without spending several hours each day in back-to-back synchronous meetings.

Maya discovers SyncUp — a comprehensive full-stack application designed to synchronize remote operations. Intrigued by its professional design, role-based structure, and powerful features, Maya decides to implement SyncUp to enhance her team's productivity and transparency.

User Registration and Authentication

Maya registers her account on SyncUp, providing her professional description as "Senior Project Coordinator." She logs in securely using her credentials, knowing that the JWT-based authentication system keeps her workspace data private and fully encrypted.

Team Creation and Code Generation

As the initiator, Maya selects the Admin role from the Role Choice page. She creates a new workspace named "Alpha-App-Dev" and the system automatically generates a unique, permanent 6-digit team code. She shares this code with her team members, allowing them to bypass complex onboarding procedures and join the specific workspace instantly.

Role-Based Team Setup

The developers join as Users (Members), while her sub-leads join as Team Leads. Maya uses her Admin Dashboard to review all access requests. She sees the name and professional title of each individual and manually grants access, ensuring that only verified employees enter the workspace.

Work Assignment and Visual Logs

The Team Leads utilize the Assign Task dashboard to set specific daily goals for each team member. They attach reference screenshots and UI wireframes as visual guides embedded within the task tickets. The Members then open these tickets, perform the assigned work, and

log their daily stand-up directly underneath. They attach images of their finished code or completed designs as visual proof of work, which Maya can review in full-screen zoom mode through the built-in Lightbox feature.

Workspace Chat and Real-Time Coordination

Instead of switching to a separate messaging application, the team uses the built-in Workspace Chat feature. Maya can send an immediate message to the live team to resolve an urgent blocker identified in a stand-up log. Because the chat is powered by WebSockets through Socket.io, all communication is instant and persistent, ensuring the team remains synchronized even while working in different time zones.

Team Feed and Historical Analysis

Maya visits the SyncUp Feed to access a structured and chronological history of the entire project. She can see every update, every work image, and every blocker reported since the team's inception. This persistent history allows her to generate accurate weekly status reports for company stakeholders with complete precision.

Maya's Experience

Thanks to SyncUp, Maya has reduced her daily meeting time by over 70%. She can now manage software development cycles more effectively by focusing on high-level strategy and project planning rather than chasing individual status updates. SyncUp has become an indispensable tool in Maya's professional workflow, helping her achieve her organizational goals with confidence, ease, and a perfectly organized digital workspace.

SYSTEM REQUIREMENTS

To ensure smooth development, deployment, and usage of the SyncUp web application, certain system prerequisites must be met. These requirements are categorized into software, setup, and hardware specifications, all of which contribute to building a robust, scalable, and professional communication platform.

1. Software Requirements

These are the essential tools and platforms required to develop, test, and run the SyncUp application efficiently.

- **Operating System:** Windows 10/11, macOS, or Linux — Supports cross-platform development and testing environments.
- **Node.js (v18 or above):** Provides the runtime environment for building the frontend logic and powers the server-side logic while handling complex API routing.
- **npm (v9 or above):** The primary package manager required to install and manage all dependencies for the MERN stack and related libraries.
- **React.js:** A powerful JavaScript library used for building the dynamic, responsive "Executive Linen" user interface with component-based architecture.
- **Express.js:** A lightweight and efficient web framework used for building the RESTful API that handles user authentication and team workspace data management.
- **MongoDB:** A NoSQL document database used to store all persistent data related to user profiles, team workspace structures, task tickets, stand-up logs, and chat messages.
- **Socket.io:** A specialized real-time communication library used to enable bi-directional WebSocket connections for the Workspace Chat feature.
- **Tailwind CSS:** A utility-first CSS framework used to achieve the unique, professional, and clean nude-themed design across all pages.

- **Axios:** A promise-based HTTP client used for making API requests from the React frontend to the Express backend.
- **Mongoose:** An Object Data Modeling (ODM) library for MongoDB and Node.js, used to define schemas and manage database operations.
- **Bcrypt.js:** A library used for hashing and salting user passwords to ensure secure credential storage.
- **JSON Web Token (JWT):** Used for stateless authentication, generating and verifying secure tokens to protect all private API endpoints.
- **Browser:** Google Chrome or Microsoft Edge (Latest Version) — For real-time rendering, Web UI testing, and high-resolution image visualization through the Lightbox feature.
- **Visual Studio Code:** The preferred Integrated Development Environment (IDE) with built-in terminal support for managing the MVC file structure.

2. Hardware Requirements

The following specifications describe the minimum and recommended hardware needed to support the development and local hosting of the SyncUp application.

- **Processor:** Intel Core i5 (8th Generation or above) or AMD Ryzen 5 or better — Ensures fast compilation of React scripts and smooth multitasking during full-stack development.
- **RAM:** Minimum 8 GB (16 GB Recommended) — Required for handling Node.js development servers, MongoDB background processes, and multiple browser instances simultaneously.
- **Storage:** At least 2 GB of free disk space — Required for the extensive node_modules dependencies, local database storage, uploaded image data, and project assets.
- **Network:** Stable Internet Connection — Mandatory for real-time WebSocket testing, API calls, npm package downloads, and cloud MongoDB connections.
- **Display:** 1366 x 768 resolution or higher — Recommended for an optimal coding experience and precise visualization of the professional application layout.

PROJECT ARCHITECTUE

Technical Architecture Description

The SyncUp application follows a structured client-server architecture built on the MERN stack (MongoDB, Express.js, React.js, and Node.js). The following description outlines the flow of data and interaction between the core components of the system.

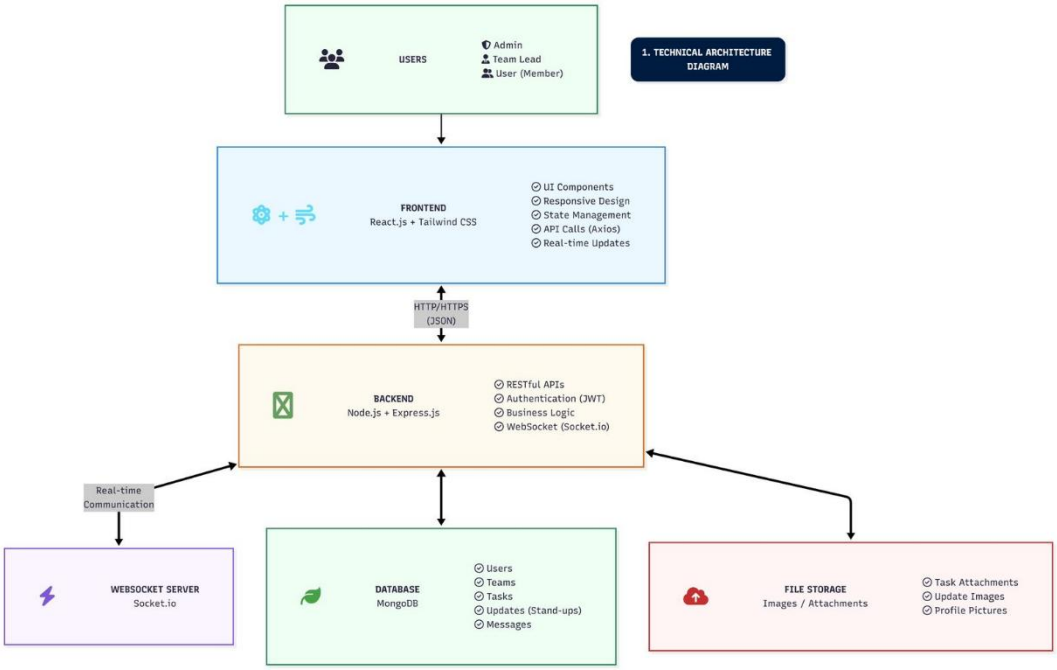
In this architecture, the flow begins with a User — which includes Administrators, Team Leads, and Team Members — interacting with the SyncUp Frontend. The user performs an action through the interface, such as submitting a structured daily stand-up form, sending a real-time message, or generating a new team workspace code.

The SyncUp Frontend captures these interactions and then transmits an asynchronous HTTP request via the Axios library, or a real-time event via the Socket.io client, to the SyncUp Backend. The backend, following the MVC (Model-View-Controller) pattern, acts as the central logic engine. It processes the incoming request, performs security verification through the JWT Middleware, and validates the associated data based on the authenticated user's role.

The backend communicates directly with the MongoDB Database to store all workspace activity securely and persistently. This database serves as the permanent ledger for the application, storing encrypted user credentials, unique team codes, task tickets, and the complete history of stand-up submissions to ensure that no project data is ever lost.

Additionally, to support the "Work Proof" feature, the backend utilizes a File Storage mechanism to handle image attachments associated with stand-up logs. This system stores multiple uploaded images and work screenshots as Base64 data strings, allowing them to be retrieved and displayed within the Team Feed for managerial review.

Once the backend has successfully processed the logic and confirmed data persistence, it sends a structured JSON response back to the SyncUp Frontend. The frontend then dynamically updates the view — such as rendering a new entry in the Team Feed, zooming an image in the Lightbox, or updating the chat log — to display the result to the user in real time.



ER Diagram Description

The Entity-Relationship (ER) diagram for SyncUp represents the various data entities and their complex interrelationships within the remote team management system. Below is a detailed description of the key components.

1. Entities

- **User:** Represents all individuals in the system, including Admins, Team Leads, and Members. Key attributes include User_ID, Name, Email, Password (Bcrypt hashed), Professional_Title, Role (Admin / TeamLead / User), and Status (Pending / Approved).

- **Team:** Represents the collaborative project workspace. Key attributes include Team_ID, Team_Name, Unique_Code (6-digit), and Admin_ID (a foreign key referencing the User who created the team).
- **Update (Stand-up Log):** Represents daily progress reports submitted by team members. Key attributes include Log_ID, Yesterday_Summary, Today_Goals, Blockers, Image_Array (Base64 strings), Timestamp, and Team_ID.
- **Task (Ticket):** Represents specific units of work assigned by Team Leads to Members. Key attributes include Task_ID, Task_Name, Description, Reference_Images, Status, and Assigned_To.
- **Message:** Represents real-time chat messages within a workspace. Key attributes include Message_ID, Sender_Name, Text_Content, Timestamp, and Team_ID.

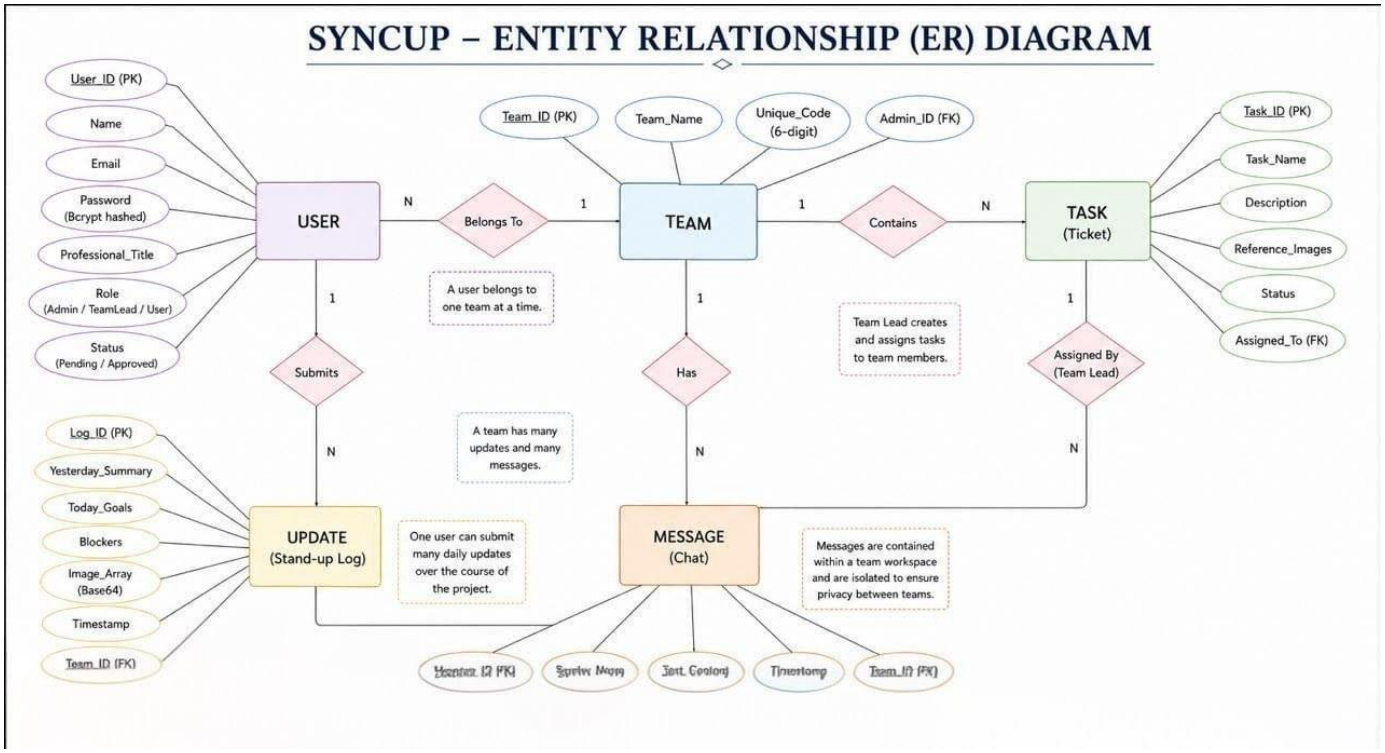
2. Relationships

- **User-Team Relationship:** Represents the affiliation of a person to a workspace. A single user belongs to one team at a time, while a team can have many users.
- **User-Update Relationship:** Indicates which member submitted a specific stand-up log. One user can submit many daily updates over the course of the project.
- **Lead-Task Relationship:** Represents the assignment logic where a Team Lead creates and assigns task tickets to specific team members.
- **Team-Message Relationship:** Indicates that real-time chat messages are contained within and isolated to a specific team workspace, ensuring data privacy between teams.

3. Primary and Foreign Keys

- **Primary Keys:** Each entity has a unique primary key generated via MongoDB ObjectId (e.g., User_ID for the User entity, Team_ID for the Team entity).
- **Foreign Keys:** Foreign keys establish relational links between entities. For example, the Update entity contains a Team_ID foreign key to ensure that stand-up history is filtered and displayed exclusively to the correct workspace members.

Overall, the ER diagram for SyncUp provides a clear visual representation of the database schema, illustrating how users, teams, tasks, daily logs, and messages are interconnected to ensure data integrity and historical persistence across the platform.



KEY FEATURES

The key features of the SyncUp application are engineered specifically for high-performance distributed teams. The platform focuses on eliminating meeting fatigue and improving team transparency through the following core functionalities:

- **Unified Authentication and Professional Profiles:** Allows users to create secure accounts using any valid email address. During the sign-up process, users provide a descriptive professional role (e.g., "Senior Developer" or "UI/UX Designer") to personalize their identity within the team workspace.
- **Role-Based Access Control (RBAC):** The system dynamically adapts the interface and available actions based on the authenticated user's role — Admin, Team Lead, or Member — ensuring that each user only has access to the tools and data appropriate for their position.
- **Admin Approval Workflow:** To maintain workspace security and integrity, the Admin must manually review and approve all users who attempt to join via the 6-digit team code before they can view or interact with any sensitive team data.
- **Structured Stand-up Logging:** Provides a three-part structured form (Yesterday's Work, Today's Goals, and Blockers) that enforces clear and consistent communication. The form also supports Multi-Image Upload, enabling team members to attach multiple screenshots as visual proof of their daily progress.
- **High-Definition Lightbox Viewer:** Enables users, leads, and admins to click on any work-proof image within the Team Feed to open it in a full-screen, zoomable Lightbox view for detailed inspection and review.
- **Real-Time Workspace Chat:** A dedicated and persistent in-application chat feature powered by WebSockets (Socket.io) that allows for instant, seamless team coordination without page refreshes or the need for external messaging tools.
- **Persistent Team Feed:** Maintains a complete and chronological history of all team stand-up updates and work submissions, ensuring that no project information is lost and that the full project history is always available for review and reporting.

- **Task Ticket Management:** Team Leads can create detailed work tickets with descriptions and reference images, and assign them directly to specific team members, providing a clear and accountable framework for daily task execution.

ROLES AND RESPONSIBILITIES

SyncUp is built around a structured, three-tier role-based system. Each role carries a distinct set of responsibilities that together ensure the smooth operation of the remote workspace.

User (Team Member)

- **Registration and Setup:** Members are responsible for creating an account with a professional title and joining the correct team by entering the 6-digit code provided by the Admin.
- **Stand-up Submission:** Users must accurately log their daily progress using the three-part stand-up form, clearly highlighting any blockers that may be preventing them from completing their assigned tasks.
- **Work Verification:** Members are responsible for attaching visual proof — in the form of screenshots or output images — to their daily stand-up submissions to provide complete transparency to the Team Lead and Admin.
- **Task Execution:** Users are expected to open their assigned task tickets, review the requirements and reference images provided by the Team Lead, and execute the work accordingly.
- **Communication:** Members utilize the Workspace Chat to coordinate with peers, ask questions, and stay updated on all team-wide changes and decisions in real time.

Team Lead

- **Task Management:** Team Leads are responsible for using the Assign Work dashboard to create detailed task tickets for team members, including comprehensive descriptions and reference imagery.

- **Progress Monitoring:** Leads regularly review the SyncUp Feed to assess team velocity, track individual contributions, and identify members who are blocked or falling behind.
- **Blocker Resolution:** Team Leads act as the first point of contact for resolving technical or process blockers identified in team members' stand-up logs, using the Workspace Chat to communicate solutions promptly.
- **Team Coordination:** Acts as the operational bridge between the Admin and the team Members, ensuring that the overall project workflow remains synchronized and on track.

Admin

- **Workspace Management:** The Admin is responsible for creating the team workspace, defining the workspace name, and managing the unique 6-digit team access code that is distributed to team members.
- **Access Approval:** Manages the security of the workspace by reviewing all incoming access requests and manually approving or rejecting Members and Leads before they gain access to the team's data and communications.
- **System Integrity:** Oversees the overall state of the workspace, including the ability to switch between multiple teams or leave a team, ensuring that user statuses and role assignments are correctly updated in the database at all times.

USER FLOW

The following describes the step-by-step interaction flow for each user role within the SyncUp application.

Member Flow

- **Auth:** Login or Sign Up and set a professional description to personalize the account profile.
- **Setup:** Select the User (Member) role and enter the 6-digit team code provided by the Admin to request access to the workspace.
- **Wait:** Remain on a pending screen and await the Admin's "Grant Access" approval before gaining entry to the workspace.
- **Execute:** Access the User Dashboard, open the assigned task ticket, fill in the three-part stand-up log (Yesterday, Today, Blockers), upload work-proof images, and submit the update.
- **Collaborate:** View the Team Feed to track all team activity, use the Lightbox to zoom into submitted work images, and coordinate with teammates through the Workspace Chat.

Team Lead Flow

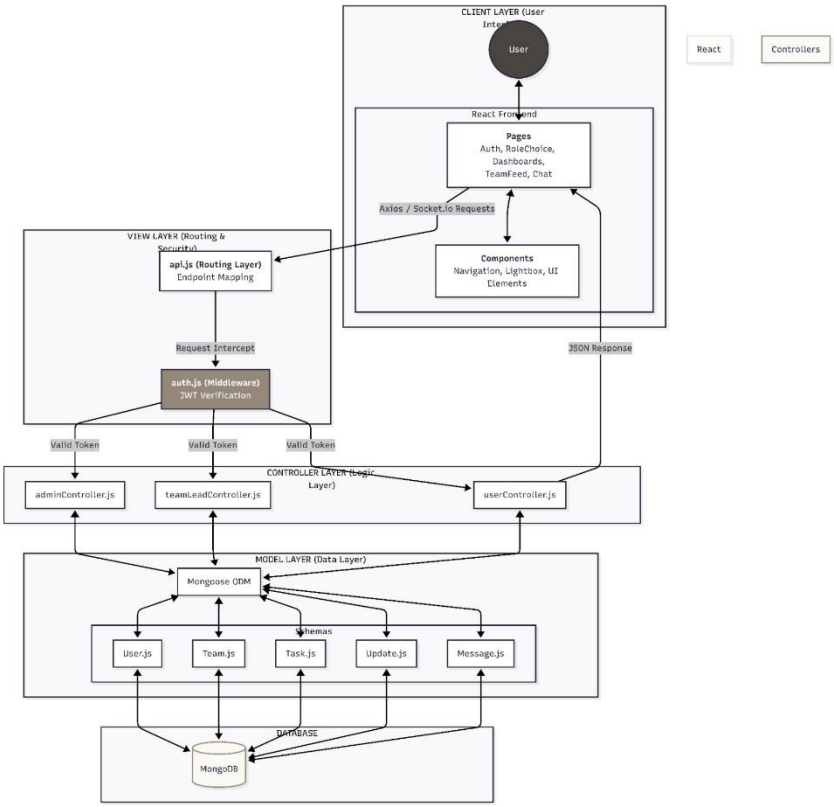
- **Auth:** Login or Sign Up and access the unified portal.
- **Setup:** Select the Team Lead role and enter the 6-digit code to join the workspace. Await Admin approval.
- **Manage Tasks:** Access the Lead Dashboard and use the Assign Work form to create task tickets with detailed descriptions and reference images, assigning them to specific team members.
- **Monitor:** Review the Team Feed regularly to track team progress and identify blockers. Use the Workspace Chat to resolve any issues promptly.

Admin Flow

- **Auth:** Login or Sign Up and access the unified Admin portal.
- **Setup:** Select the Admin role, enter the team name, and click "Generate Code" to create the unique 6-digit workspace access code to share with the team.
- **Manage:** Access the Admin Dashboard to view all pending Access Requests and manually grant or deny access to Users and Team Leads.
- **Control:** Switch between multiple team workspaces or disconnect from the current workspace using the Dashboard dropdown menu as needed.

MVC PATTERN

The SyncUp application follows the Model-View-Controller (MVC) architectural pattern — a software design approach that separates the application into three distinct, interconnected layers. This separation is fundamental to creating a professional, enterprise-grade tool, as it allows for high modularity, easier long-term maintenance, and seamless scalability as team sizes and feature requirements grow.



Model Layer (Data Layer)

The Model layer is responsible for handling all data-related logic and serves as the structural blueprint for the application's information. In SyncUp, this includes the definition of data schemas for Users, Teams, Tasks, Updates (Stand-up Logs), and Messages. These models are implemented using Mongoose, which provides a schema-based solution for modeling application data in MongoDB. The Model layer handles input validation, default value assignment (such as automatic timestamps), and the relational mapping between users and their specific team workspaces.

Controller Layer (Logic Layer)

The Controller layer acts as the "brain" of the application, serving as the intermediary between the View (routing) layer and the Model layer. It receives incoming requests from the frontend — such as a Team Lead assigning a new work ticket or a Member submitting a daily stand-up update — processes the input data (including Bcrypt password hashing or Base64 image transformation), calls the appropriate Mongoose model methods, and returns a structured JSON response to the client.

Byseparating the controllers into role-specific files (adminController.js, teamLeadController.js, userController.js), the business logic of the application remains well-organized, readable, and easy to maintain.

View Layer (Routing Layer)

In the context of the SyncUp backend REST API, the View layer is implemented as the Routing Layer. This is where all API endpoints are defined within the api.js file. These endpoint definitions determine how the backend responds to different HTTP requests (GET and POST) and are responsible for invoking the correct controller functions. This layer also incorporates the JWT Middleware as a compulsory guard to verify authentication tokens

before permitting a user to access protected resources such as the Team Feed or the Workspace Chat.

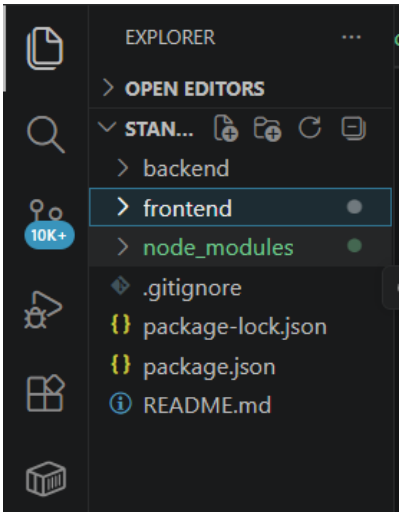
Advantages of Using MVC in SyncUp

- **Separation of Concerns:** Each layer has a clearly defined and singular responsibility. The React UI stays focused on presentation and user experience, while the Node/Express layer handles business logic, and MongoDB manages data persistence independently, greatly improving overall code readability and maintainability.
- **Scalability:** New features — such as the real-time chat application or the multiple-image upload system — can be added easily by creating new routes, controllers, and models without breaking or disrupting any existing core functionality.
- **Reusability:** Logic defined within the controllers and models can be reused across different parts of the application. For example, the JWT authentication logic used for the initial user login is also reused to verify permissions when an Admin attempts to grant access to a new workspace member.
- **Independent Testing:** Each layer can be tested in complete isolation. The API endpoints can be verified using tools such as Postman to ensure that the JWT middleware and all controller functions are behaving correctly, even before the React frontend has been fully developed or connected.
- **Collaboration-Friendly:** The MVC structure allows multiple developers to work on the project simultaneously and without code conflicts. One developer can focus on building the nude-themed UI components in the React Pages folder, while another independently optimizes the database queries within the Mongoose Models.

PROJECT SETUP AND CONFIGURATION

Creating the Project Folder

1. Create a new root folder named SyncUp-Project.
2. Inside that root folder, create two sub-folders.
3. Name one sub-folder Client (for the React frontend).
4. Name the other sub-folder Server (for the Node.js/Express backend).
5. Open the root SyncUp-Project folder in Visual Studio Code.



Client Setup (Installing the React App)

6. Open the Client folder in the integrated terminal of VS Code.
7. Run the following command to scaffold the project using Vite: npm create vite@latest . -- --template react
8. Select React as the framework and JavaScript as the language variant from the interactive options.
9. Run npm install to load all base packages and dependencies.
10. Install the additional professional tools required for this project: npm install axios react-router-dom lucide-react socket.io-client
11. To start the React development server, type: npm run dev

Server Setup (Node.js Initialization)

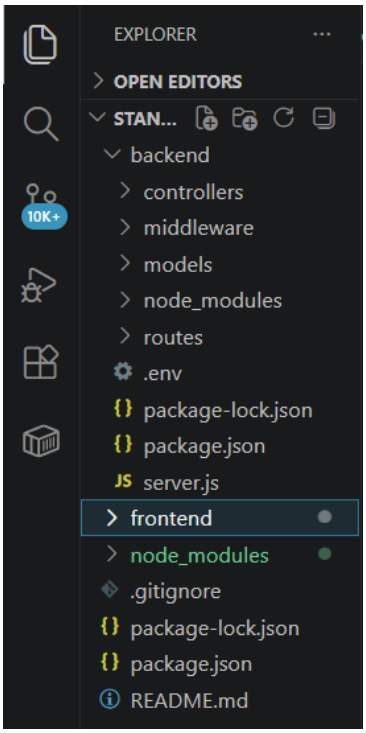
12. Open the Server folder in the integrated terminal of VS Code.
13. Type npm init -y to automatically generate the package.json configuration file.
14. Install all necessary backend packages: npm install express mongoose cors bcryptjs jsonwebtoken socket.io dotenv
15. Create the main entry file: server.js
16. Create the full MVC folder structure inside the Server folder: models, controllers, routes, and middleware.

BACKEND DEVELOPMENT

The SyncUp backend is a RESTful API server built with Node.js and Express.js, structured following the MVC pattern. It manages all server-side logic, database communication, file storage, and real-time WebSocket events.

Backend Reference Video:

https://drive.google.com/drive/folders/1lBPaqF_21CoM1ZF1jWqauxc1zKMCax0W?usp=sharing



Backend Structure

- **Controllers:** Contains the role-specific business logic for each type of user.
- **Middleware:** Houses the auth.js guard for JWT token verification and request protection.
- **Routes:** Defines all API endpoint mappings within the api.js file.
- **Server.js:** The application's entry point that configures Express, establishes the MongoDB connection, and initializes real-time WebSocket communication via Socket.io.

Controllers

• UserController.js: Acts as the central controller that integrates all core functionalities of the system, including unified authentication (Login and Sign-up), session handling, and validation. It manages the complete workspace lifecycle by enabling team creation, generating unique 6-digit access codes. It also supports essential workspace operations such as joining teams, leaving, and switching between multiple workspaces.

It further incorporates task and activity management features, allowing users to assign work, create tickets with multiple reference images, and monitor progress through a live Team Feed. Additionally, it handles daily stand-up log submissions with multi-image support, while ensuring proper validation, error handling, and efficient coordination of all user, team, and task-related operations within a single unified module.

DATABASE DEVELOPMENT

1. Configure MongoDB

- **Install Mongoose:** In your Server project directory, run: npm install mongoose
- **Establish Connection:** Utilize the .env file to securely store the MONGO_URI environment variable and connect the MVC engine to either a local or cloud-hosted MongoDB instance.

Create Database Connection

The database connection is established asynchronously to ensure that the Express server only begins accepting requests once the MongoDB connection has been successfully confirmed. A professional try-catch block is used to handle all connection errors gracefully and log the connection status to the terminal.

Create Schemas

The database schemas are designed based on the relational Entity-Relationship logic defined in the ER diagram to ensure complete data integrity and consistency across the entire platform. The following schemas are implemented using Mongoose:

17. **User.js:** Schema for persistent user accounts, including the user's name, email, Bcrypt-hashed password, professional title, assigned role (Admin / TeamLead / User), and workspace status (Pending / Approved).

```
backend > models > JS User.js > ...
const mongoose = require('mongoose');
const UserSchema = new mongoose.Schema({
    name: { type: String, required: true },
    email: { type: String, unique: true, required: true },
    password: { type: String, required: true },
    role: { type: String, default: 'Team Member' },
    teamId: { type: mongoose.Schema.Types.ObjectId, ref: 'Team', default: null }
});
module.exports = mongoose.models.User || mongoose.model('User', UserSchema);
```

18. **Team.js:** Schema for unique team workspaces, storing the team name, the Admin's user ID, and the permanent 6-character alphanumeric access code.

```
backend > models > JS Team.js > ...
const mongoose = require('mongoose');
const TeamSchema = new mongoose.Schema({
    name: { type: String, required: true },
    code: { type: String, unique: true, required: true },
    adminId: { type: mongoose.Schema.Types.ObjectId, ref: 'User' }
});
module.exports = mongoose.models.Team || mongoose.model('Team', TeamSchema);
```

1. **Task.js:** Schema for work tickets assigned by Team Leads, supporting a task name, detailed description, assigned member ID, status tracking, and an array of reference image strings.

```
backend > models > JS Task.js > ...
const mongoose = require('mongoose');

const TaskSchema = new mongoose.Schema({
    teamId: { type: mongoose.Schema.Types.ObjectId, ref: 'Team' },
    assignerName: String,
    assigneeName: String,
    taskName: String, // <--- IDENTICAL NAME
    description: String,
    images: [String],
    createdAt: { type: Date, default: Date.now }
});

module.exports = mongoose.models.Task || mongoose.model('Task', TaskSchema);
```

2. **Update.js:** Schema for daily stand-up logs, storing the submitting user's ID, yesterday's summary, today's goals, any blockers reported, an array of work-proof image Base64 strings, and an automatic timestamp.

```
backend > models > JS Update.js > ...
const mongoose = require('mongoose');
module.exports = mongoose.model('Update', new mongoose.Schema({
    userId: mongoose.Schema.Types.ObjectId,
    userName: String,
    teamId: mongoose.Schema.Types.ObjectId,
    taskName: String,
    yesterday: String,
    today: String,
    blockers: String,
    images: [String], // <--- This MUST be an array of strings
    date: { type: String, default: () => new Date().toLocaleDateString() },
    createdAt: { type: Date, default: Date.now }
}));
```

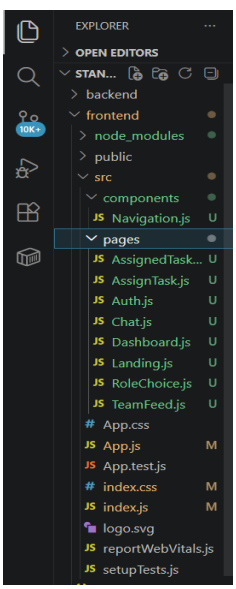
3. **Message.js:** Schema for persistent real-time chat messages within a workspace, storing the sender's name, message text content, the associated team ID, and a timestamp.

```
backend > models > JS Message.js > ...
const mongoose = require('mongoose');

const MessageSchema = new mongoose.Schema({
    teamId: { type: String, required: true }, // Changed to String to prevent ObjectId crashes
    senderId: { type: String, required: true },
    senderName: { type: String, required: true },
    text: { type: String, required: true },
    createdAt: { type: Date, default: Date.now }
});

module.exports = mongoose.models.Message || mongoose.model('Message', MessageSchema);
```

FRONT-END DEVELOPMENT

The SyncUp frontend is built with React.js using the Vite build tool and styled exclusively with Tailwind CSS to achieve the professional "Executive Linen" nude palette. The application is structured using a clear, role-based page organization to separate concerns between different user types.

Frontend reference Video :
https://drive.google.com/drive/folders/10mQYJTkZSdiqrsjmjvS-hOkWu4a2HgwA?usp=sharing



Frontend Structure

- **Pages (src/pages):** Main view components representing each distinct screen in the application.
- **Components (src/components):** Reusable UI elements shared across multiple pages.
- **Assets:** Holds global CSS configurations for the "Executive Linen" nude palette and shared design tokens.

Role-Based Pages (src/pages)

- **Auth.js:** A unified, clean interface for both Sign-in and Sign-up, featuring action-based placeholders, a professional role description input, and seamless switching between the two forms.
- **RoleChoice.js:** The central navigation hub where authenticated users select their role (Admin, Lead, or Member) and perform the appropriate team setup or join actions for their chosen workspace.
- **AdminDashboard.js:** A specialized management view exclusively for Admins, displaying all pending access requests with each applicant's name and professional title, and providing the ability to approve or reject each request.
- **Chat.js:** A dedicated real-time communication portal that provides an always-on messaging interface for the team, powered by WebSockets for instant message delivery without page refresh.
- **Dashboard.js**: UserDashboard.js: Acts as a unified interface for all roles, combining administrative, team lead, and member functionalities into a single dashboard. It displays workspace details, access to task management features such as creating and assigning work, uploading multiple reference images, and managing task workflows efficiently.

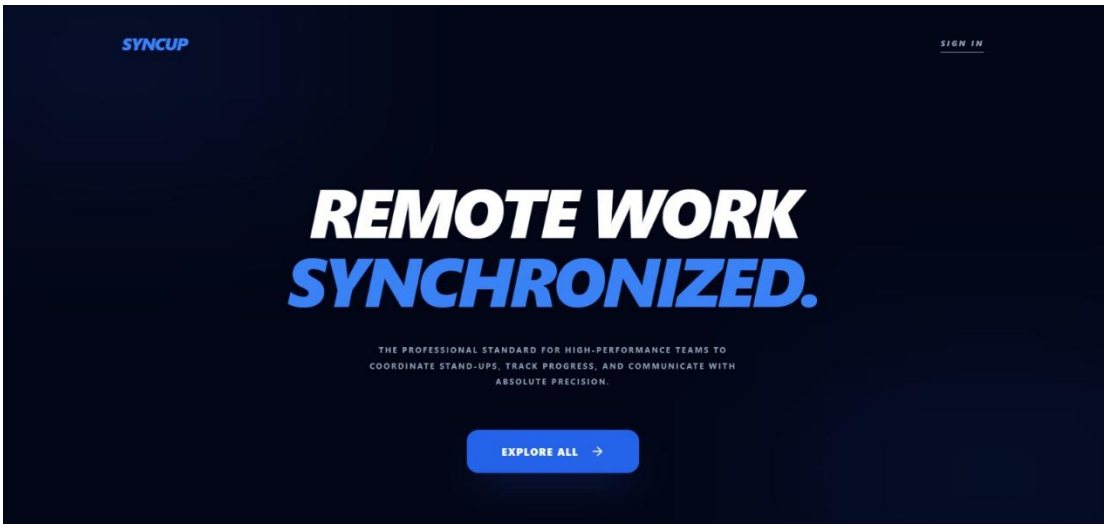
Components (Reusable Parts)

- **Navigation.js:** A backdrop-blur navigation bar featuring the SyncUp branding and a professional dropdown menu containing the "Switch Teams" and "Leave the Team" options for workspace management.
- **Lightbox.js:** A reusable full-screen image overlay component that renders when a user clicks on any work-proof image in the Team Feed, providing a zoomable, high-definition view.
- **ui/:** Contains a collection of styled UI containers and nude-themed button components used consistently throughout the application to maintain a professional and visually coherent look and feel.

OUTPUT SCREENSHOTS

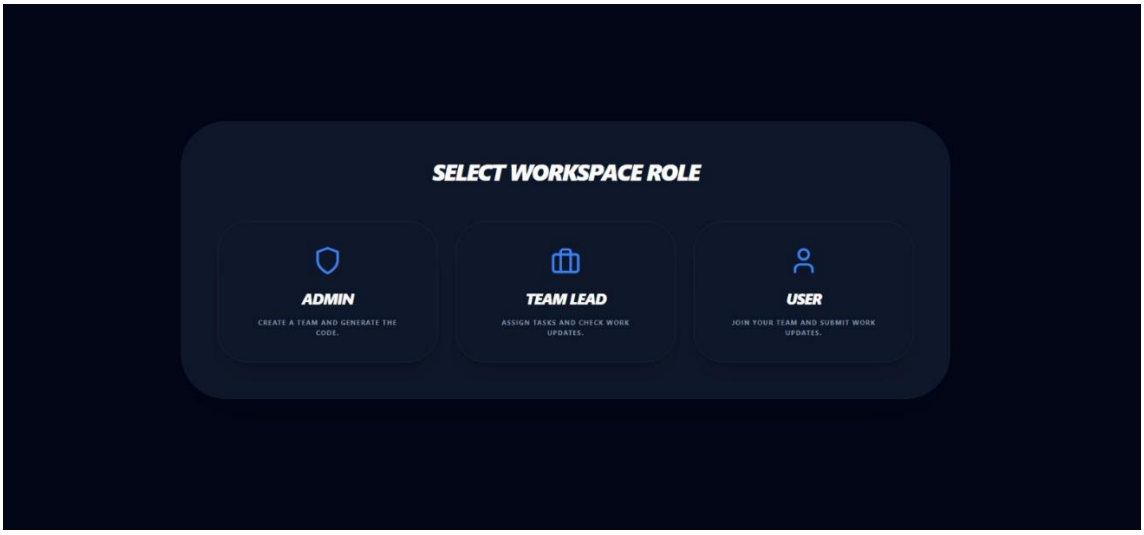
The following screenshots illustrate the key screens and functional outputs of the SyncUp web application, demonstrating the complete user journey from registration through daily stand-up submission and team coordination.

1. Landing Page

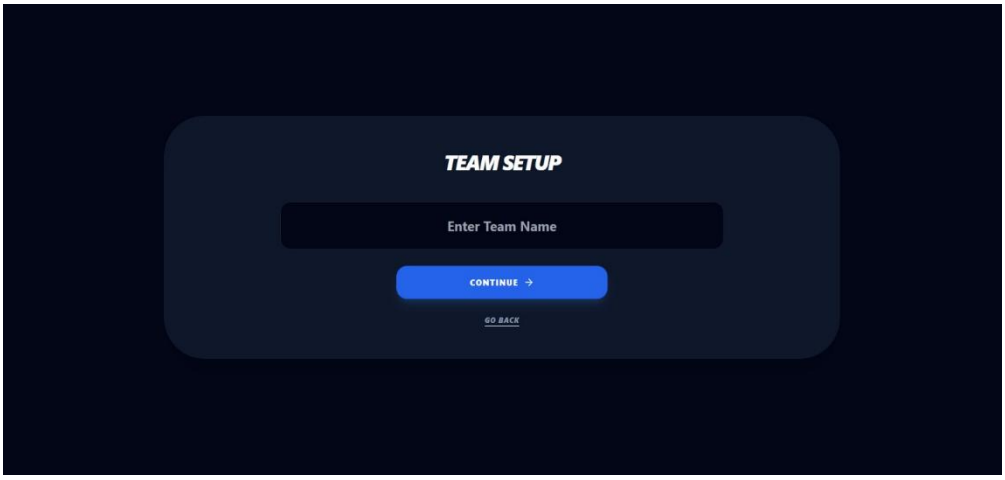


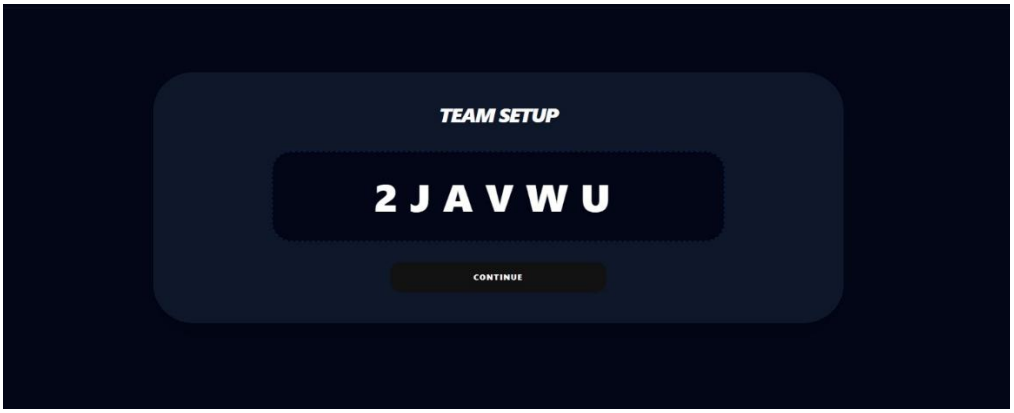
2. Authentication Page (Sign Up)

3. Role Choice Page

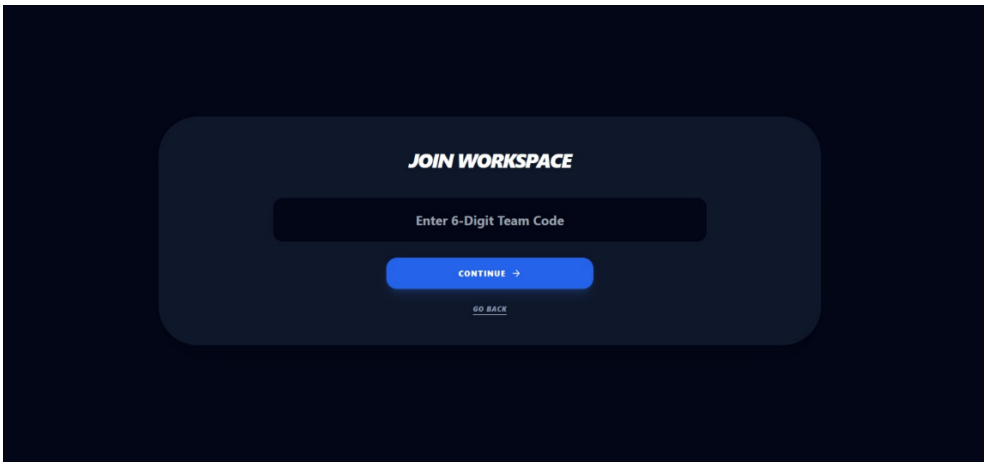


4. Admin Workspace Setup

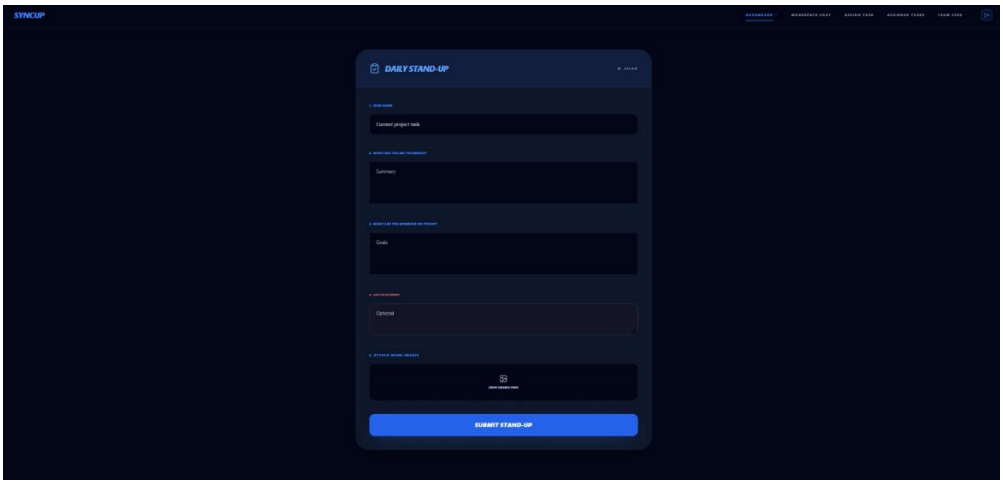




5. User / Lead Workspace Joining Page

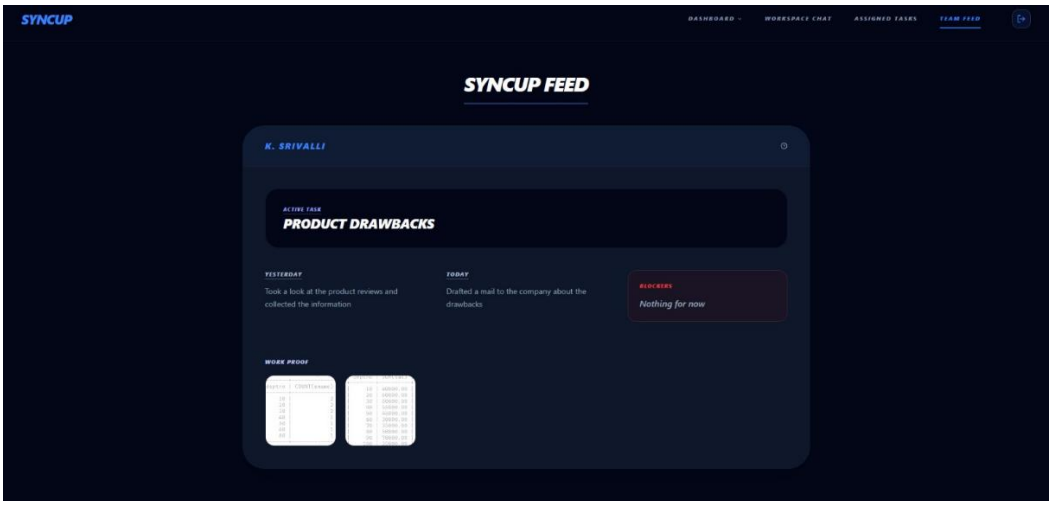


6. User Dashboard (Daily Stand-up Log)

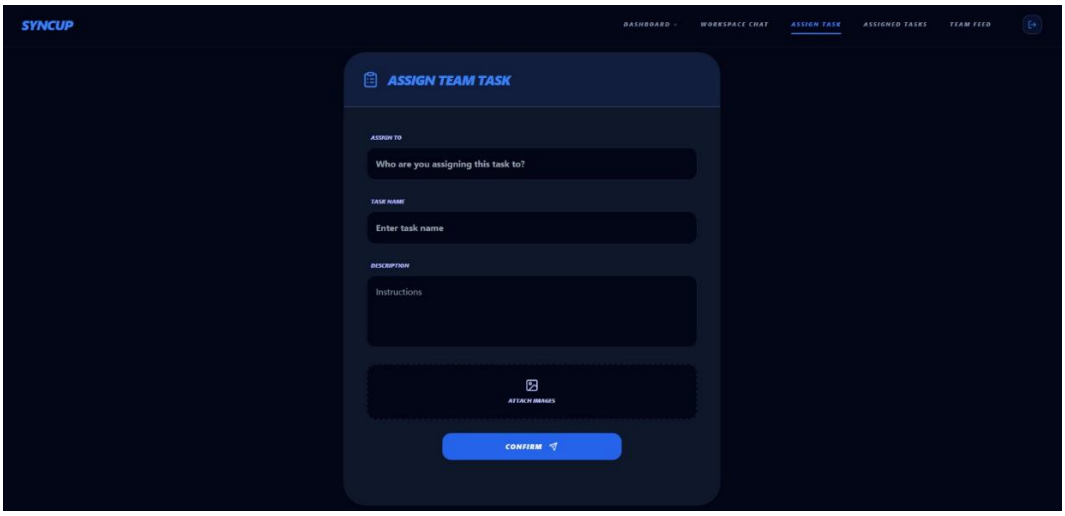


8. SyncUp Team Feed (Empty Space)

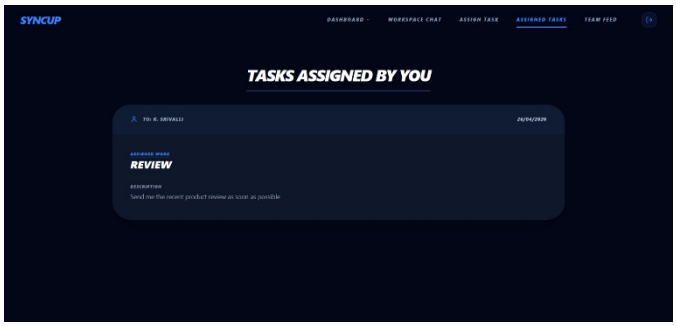
9. SyncUP Team Feed (Updated Space)

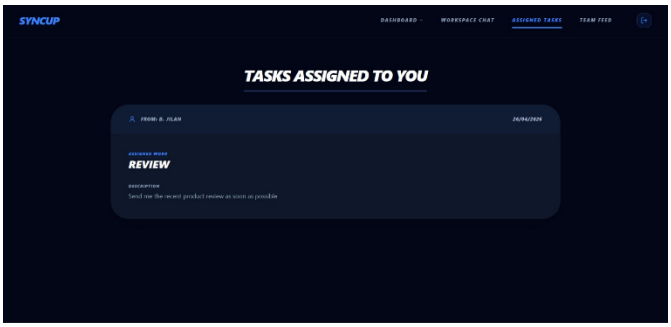


10. Assign work Page

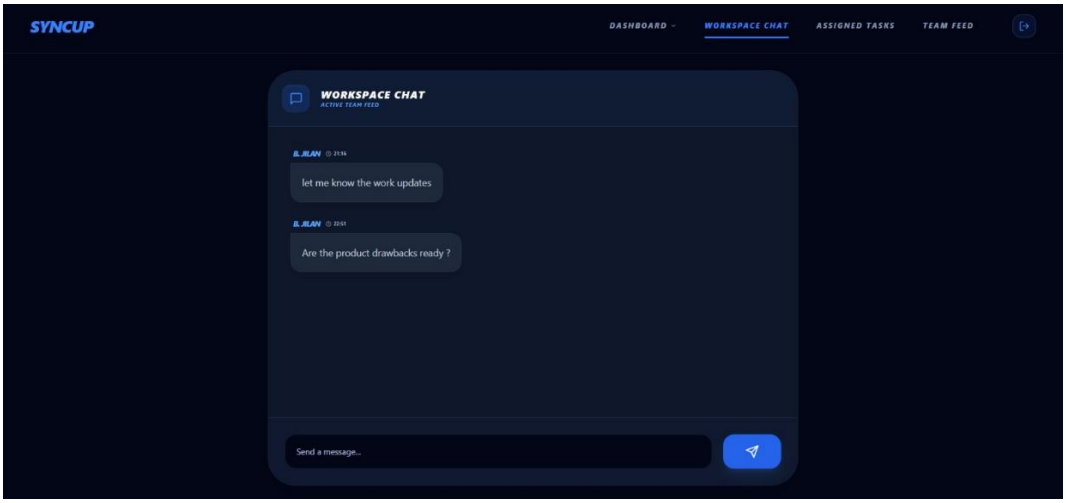


11. Assigned Work Page (Team Lead and User)

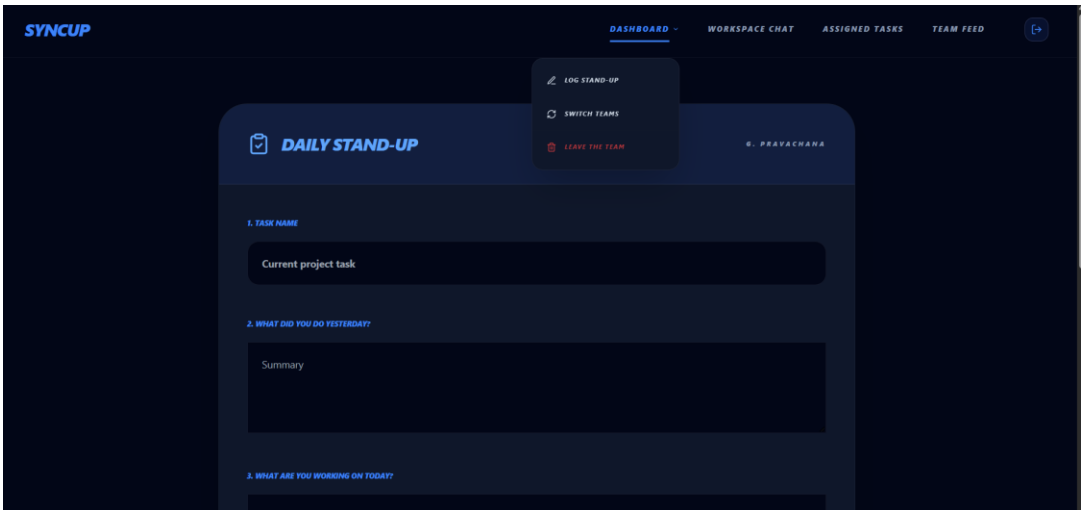




12. Workspace Chat



13. Navigation Dropdown Menu



Demo Video Link:

https://drive.google.com/drive/folders/1ElAjqlwyBz926ga_NK6Aol6B8OSjnTNE?usp=sharing

Code Drive Link:

CONCLUSION

SyncUp successfully addresses one of the most persistent and costly challenges in modern distributed software development — the inefficiency of synchronous daily meetings in remote, cross-time-zone teams. By providing a structured, role-based, and asynchronous platform for daily progress reporting, SyncUp empowers teams to communicate clearly, hold each other accountable, and maintain complete project transparency without sacrificing productive working hours to recurring stand-up calls.

The application was developed using the MERN stack (MongoDB, Express.js, React.js, and Node.js), following the Model-View-Controller (MVC) architectural pattern to ensure a clean separation of concerns, high modularity, and straightforward scalability. Key technical implementations include JWT-based stateless authentication for secure, role-specific access control; Bcrypt password hashing for robust credential security; Socket.io-powered WebSockets for real-time team communication; and Base64-encoded multi-image storage for persistent, visual work verification within stand-up logs.

The three-tier role system — comprising the Admin, Team Lead, and Team Member — mirrors real-world organizational hierarchies and provides each role with precisely the tools and permissions needed to fulfill their responsibilities effectively. The Admin controls workspace access and security, the Team Lead manages task assignment and progress monitoring, and the Team Member executes work and submits structured daily logs with visual proof.

Features such as the high-definition Lightbox image viewer, the persistent Team Feed, the built-in Workspace Chat, and the Admin Approval workflow collectively create an ecosystem that rivals far more complex project management solutions, while maintaining an accessible and professionally polished user experience through the "Executive Linen" design language.

In conclusion, SyncUp demonstrates that thoughtfully designed full-stack web applications — built on industry-standard technologies and sound architectural principles — can profoundly improve team productivity, reduce operational overhead, and provide management-level visibility into distributed software projects. The application stands as a solid foundation upon which future enhancements — such as mobile applications, analytics dashboards, calendar integrations, and AI-powered blocker suggestions — can be confidently built.

REFERENCES

- MongoDB Documentation. (2024). MongoDB Manual. https://www.mongodb.com/docs/
- Express.js Documentation. (2024). Express — Node.js web application framework. https://expressjs.com/
- React Documentation. (2024). React — The library for web and native user interfaces. https://react.dev/
- Socket.io Documentation. (2024). Socket.IO — Bidirectional and low-latency communication. https://socket.io/docs/
- Mongoose Documentation. (2024). Mongoose ODM v8. https://mongoosejs.com/docs/
- JSON Web Tokens. (2024). Introduction to JSON Web Tokens. https://jwt.io/introduction/
- Tailwind CSS Documentation. (2024). Tailwind CSS — A utility-first CSS framework. https://tailwindcss.com/docs/
- Vite Documentation. (2024). Vite Next Generation Frontend Tooling. https://vitejs.dev/
- Node.js Documentation. (2024). Node.js v18 LTS Documentation. https://nodejs.org/en/docs/
- Bcrypt.js. (2024). bcryptjs — Optimized bcrypt in JavaScript. https://www.npmjs.com/package/bcryptjs/

Project Overview Video:

https://drive.google.com/drive/folders/1ElAjqlwyBz926ga_NK6Aol6B8OSjnTNE?usp=sharing

Code Drive Link:

https://drive.google.com/drive/folders/1788pMiqGEIaxk7e7tVraupz0h79yA77x?usp=sharing